



Distributed Systems

Basic PAXOS DSM Blockchain



Recap



- Optimizations
 - Koo-Toueg Coordinated Checkpointing Algorithm
 - Koo-Toueg Recovery Algorithm
- Fault tolerance using replication
- Voting – Static / Dynamic

Requirements for consensus



Safety requirements:

- Only a value that has been proposed may be chosen,
- Only a single value is chosen, and
- A process never learns that a value has been chosen unless it actually has been.

Liveness requirements:

- ensure that some proposed value is eventually chosen and,
- if a value has been chosen, then a process can eventually learn the value.

IIT ROORKEE ■ ■ ■

PAXOS



- There are three classes of agents:
 - **Proposers**
 - **acceptors**, and
 - **learners.**
- A single process may act as more than one agent
- Agents communicate with one another by sending messages.
- Model: asynchronous, non-Byzantine, in which:
 - Agents operate at arbitrary speed, may fail by stopping, and may restart.
 - Messages can take arbitrarily long to be delivered, can be duplicated, and can be lost, but they are not corrupted.

IIT ROORKEE ■ ■ ■

Algorithm



Phase 1

- (a) A proposer selects a proposal number n and sends a **prepare (PROPOSE)** request with number n to a majority of acceptors.
- (b) If an acceptor receives a prepare request with number n greater than that of any prepare request to which it has already responded, then it responds to the request with a **promise (AGREE)** not to accept any more proposals numbered less than n and with the (value of) highest-numbered proposal (if any) that it has accepted.

Algorithm



Phase 2.

- (a) If the proposer receives a response to its prepare requests (numbered n) from a majority of acceptors, then it sends an **accept (COMMIT)** request to each of those acceptors for a proposal numbered n with a value v , where v is the value of the highest-numbered proposal among the responses, or is any value if the responses reported no proposals.
- (b) If an acceptor receives an accept request for a proposal numbered n , it accepts the proposal unless it has already responded to a prepare request having a number greater than n .

Algorithm



Phase 1a: Proposer (PREPARE)

A proposer initiates a **PREPARE** message, picking a unique, ever-incrementing value.

```
ID = cnt++;
send PREPARE(ID)
```

Phase 1b: Acceptor (PROMISE)

An acceptor receives a **PREPARE(ID)** message:

```
if (ID <= max_id)
    do not respond (or respond with a "fail" message)
else
    max_id = ID      // save highest ID we've seen so far
    if (proposal_accepted == true) // was a proposal already accepted?
        respond: PROMISE(ID, accepted_ID, accepted_VALUE)
    else
        respond: PROMISE(ID)
```

Source: article By Paul Krzyzanowski, Rutgers University

IIT ROORKEE ■ ■ ■

7

Algorithm



Phase 2a: Proposer (PROPOSE)

The proposer now checks to see if it can use its proposal or if it has to use the highest-numbered one it received from among all responses:

```
did I receive PROMISE responses from a majority of acceptors?
if yes
    do any responses contain accepted values (from other proposals)?
    if yes
        val = accepted_VALUE    // value from PROMISE message with the highest accepted ID
    if no
        val = VALUE             // we can use our proposed value
    send PROPOSE(ID, val) to at least a majority of acceptors
```

IIT ROORKEE ■ ■ ■

8

Algorithm



Phase 2b: Acceptor (ACCEPT)

Each acceptor receives a PROPOSE(ID, VALUE) message from a proposer. If the ID is the highest number it has processed then accept the proposal and propagate the value to the proposer and to all the learners.

```
if (ID == max_id) // is the ID the largest I have seen so far?
    proposal_accepted = true    // note that we accepted a proposal
    accepted_ID = ID           // save the accepted proposal number
    accepted_VALUE = VALUE      // save the accepted proposal data
    respond: ACCEPTED(ID, VALUE) to the proposer and all learners
else
    do not respond (or respond with a "fail" message)
```

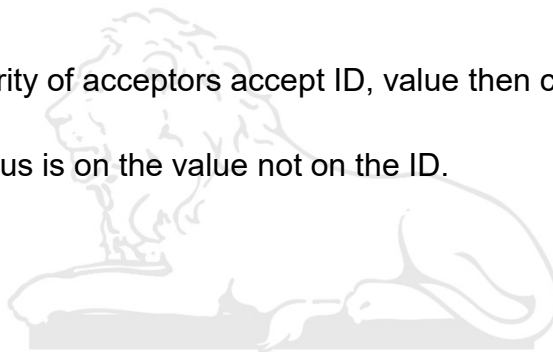
IIT ROORKEE ■ ■ ■

9

PAXOS



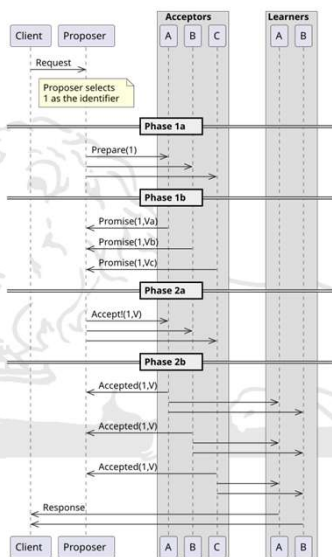
- If a majority of acceptors accept ID, value then consensus is reached.
- Consensus is on the value not on the ID.



IIT ROORKEE ■ ■ ■

10

A failure free execution of PAXOS



Source: Wikipedia

IIT ROORKEE

11

Performance Optimization



- A proposer can make multiple proposals.
- It can abandon a proposal in the middle of the protocol at any time.
- It is probably a good to abandon a proposal if some proposer has begun trying to issue a higher-numbered one.
- Therefore, if an acceptor ignores a *prepare* or *accept* request, then it should inform the proposer, who should then abandon its proposal.
- This is a performance optimization that does not affect correctness.

(Correctness is maintained, even though requests and/or responses for the proposal may arrive at their destinations long after the proposal was abandoned.)

IIT ROORKEE

12

Learning a Chosen Value



Approach 1:

- Each acceptor, whenever it accepts a proposal, respond to all learners, sending them the proposal.
- This allows learners to find out about a chosen value as soon as possible, but it requires each acceptor to respond to each learner
- Number of responses = number of acceptors * number of learners.

Approach 2:

- We can have the acceptors respond with their acceptances to a distinguished learner, which in turn informs the other learners when a value has been chosen.
- This approach requires an extra round for all the learners to discover the chosen value.
- It is also less reliable, since the distinguished learner could fail.
- Number of responses = number of acceptors + number of learners.

IIT ROORKEE ■ ■ ■

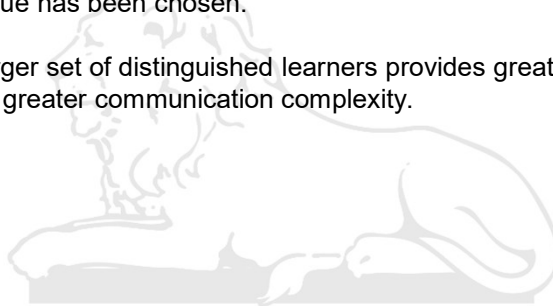
13

Learning a Chosen Value



Approach 3:

- The acceptors could respond with their acceptances to some set of distinguished learners, each of which can then inform all the learners when a value has been chosen.
- Using a larger set of distinguished learners provides greater reliability at the cost of greater communication complexity.



IIT ROORKEE ■ ■ ■

14

Progress



- A scenario in which two proposers each keep issuing a sequence of proposals with increasing numbers, none of which are ever chosen.
- Proposer p completes phase 1 for a proposal number $n1$.
- Another proposer q then completes phase 1 for a proposal number $n2 > n1$.
- Proposer p 's phase 2 *accept* requests for a proposal numbered $n1$ are ignored because the acceptors have all promised not to accept any new proposal numbered less than $n2$.
- So, proposer p then begins and completes phase 1 for a new proposal number $n3 > n2$, causing the second phase 2 *accept* requests of proposer q to be ignored.
- And so on.

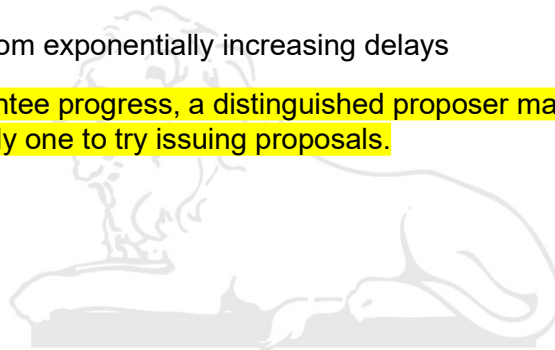
IIT ROORKEE ■ ■ ■

15

Progress



- Use random exponentially increasing delays
- To guarantee progress, a distinguished proposer may be selected as the only one to try issuing proposals.



IIT ROORKEE ■ ■ ■

16



Distributed Shared Memory



Distributed Shared Memory

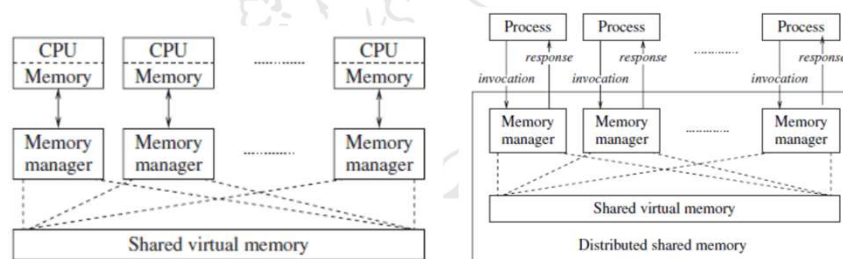


- Distributed shared memory (DSM) gives the impression of a single monolithic memory to the programmer, as in traditional von Neumann architecture.
- Programmers access the data across the network using only read and write primitives, as they would in a uniprocessor system.
- Programmers do not have to deal with send and receive communication primitives

Distributed Shared Memory



- A part of each computer's memory is earmarked for shared space, and the remainder is private memory.
- To provide programmers with the **illusion of a single shared address space**, a **memory mapping management layer** is required to manage the *shared virtual memory space*.



IIT ROORKEE

19

Advantages of DSM



- Shields programmer from Send/Receive primitives
- Single address space: simplifies passing-by-reference and passing complex data structures
- DSM uses simpler software interfaces, and cheaper off-the-shelf hardware. Hence cheaper than dedicated multiprocessor systems
- No memory access bottleneck, as no single bus
- Large virtual memory space
- DSM programs portable as they use common DSM programming interface

IIT ROORKEE

20

DSM - Issues



- But, it is a distributed system and a network, and the data needs to be shared in some fashion.
- Data may be replicated and/or there may be concurrent access to data
 - => concurrency control needs to be enforced.
- When multiple processors wish to access the same data object, a decision about how to handle concurrent accesses needs to be made.

DSM - Issues

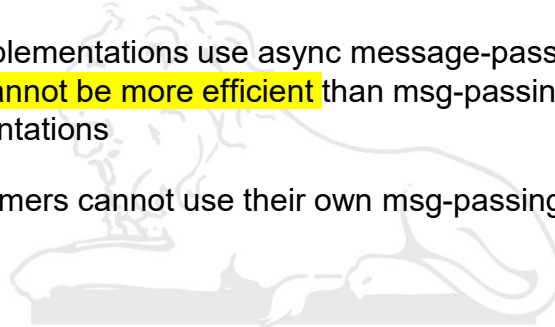


- If concurrent access is permitted by different processors to different replicas, the problem of replica consistency needs to be addressed.
- The semantics of what value a read operation returns to the program needs to be specified.
- Programmers need to understand this semantics - program logic depends on it
- Is DSM transparent to the programmer?

Disadvantages of DSM



- Programmers need to understand consistency models, to write correct programs
- DSM implementations use async message-passing, and hence **cannot be more efficient** than msg-passing implementations
- Programmers cannot use their own msg-passing solutions.



Memory consistency models



- Assume n processes and s_i memory operations / process P_i .
- Also assume that **all the operations issued by a process are executed sequentially (that is, pipelining is disallowed)**,
- There are a total of

$$(s_1 + s_2 + \dots + s_n)! / (s_1! s_2! \dots s_n!)$$

possible permutations or interleavings of the operations issued by the processes.

- Problem of ensuring **memory coherence** - problem of identifying which of these interleavings are “correct,”
- The *memory consistency model* defines the set of allowable memory access orderings.

Memory consistency models



- **Traditional definition of correctness** - a correct memory execution is one that returns to each *Read* operation, the value stored by the **most recent** *Write* operation
- What is most recent in the traditional Von Neumann architecture
- Two salient features of such a system are:
 - (i) a common global time axis is implicitly available in a uniprocessor system
 - (ii) each write is immediately visible to all processes.

Memory consistency models



- What is “**most recent**” in the presence of **concurrent access** and **multiple replicas** of the data item.
=> a clear definition of correctness is required in such a system
- Disallow inter-leavings that make no semantic sense, while not being overly restrictive so as to permit a high degree of concurrency.

Strict consistency / atomic consistency / linearizability



The model is more formally specified as follows:

1. Any *Read* to a location (variable) is required to return the value written by the most recent *Write* to that location (variable) **as per a global time reference**.

For operations that overlap as per the global time reference, following is needed

2. All operations appear to be executed atomically and sequentially.
3. All processors see the same ordering of events, **which is equivalent to the global-time occurrence of non-overlapping events**.

IIT ROORKEE

27

Strict consistency / atomic consistency / linearizability



- invocation and the response to each operation can both be separately viewed as being atomic events
- An execution sequence in global time is viewed as a sequence Seq of such invocations and responses.
- This Seq must satisfy the following conditions:

Liveness: Each invocation must have a corresponding response.

Correctness: The projection of Seq on any processor i , denoted Seq_i , must be a sequence of alternating invocations and responses if pipelining is disallowed.

IIT ROORKEE

28

Strict consistency / atomic consistency / linearizability

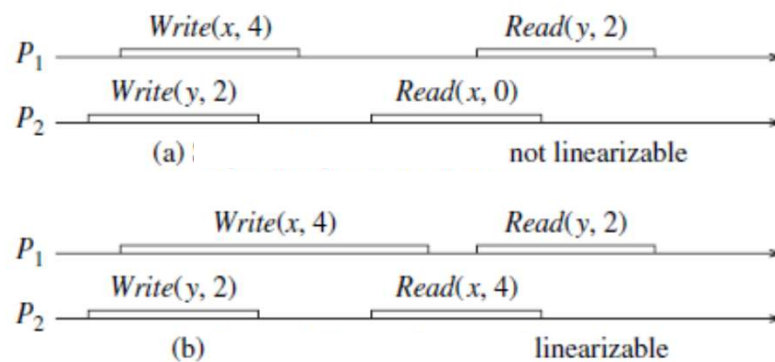


- A sequence Seq of invocations and responses is *linearizable* (LIN) if there is a permutation Seq' of adjacent pairs of corresponding <invoc, resp> events satisfying:
 - For every variable v , the projection of Seq' on v , denoted Seq' _{v} , is such that every *Read* (adjacent <invoc, resp> event pair) returns the most recent *Write* (adjacent <invoc, resp> event pair) that immediately preceded it.
 - If the response op1(resp) of operation op1 occurred before the invocation op2(invoc) of operation op2 in Seq, then op1 (adjacent <invoc, resp> event pair) occurs before op2 (adjacent <invoc, resp> event pair) in Seq.

IIT ROORKEE

29

Consistency

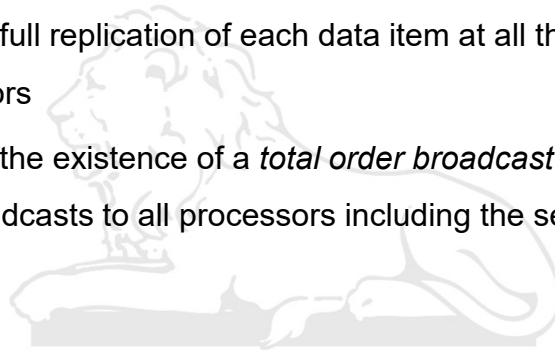


30

Implementation



- A global time scale needs to be simulated
- Assume full replication of each data item at all the processors
- Assume the existence of a *total order broadcast* primitive that broadcasts to all processors including the sender.



Implementation (The Algorithm)



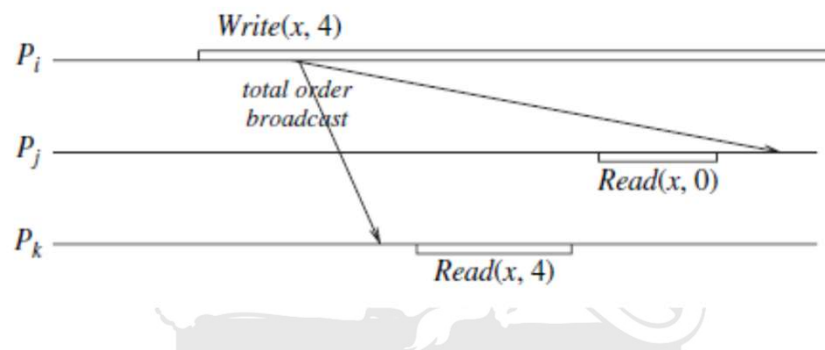
(shared var) **int**: x;

- (1) Memory manager receives a *Read* or *Write* from application:
 - (1a) **total_order_broadcast** the *Read* or *Write* request to all processors;
 - (1b) **await** own request that was broadcast;
 - (1c) **perform** pending response to the application as follows
 - (1d) **case Read**: return value from local replica;
 - (1e) **case Write**: write to local replica and return ack to application.
- (2) When the memory manager receives a **total_order_broadcast** (*Write*, x, *val*) from network:
 - (2a) **write** *val* to local replica of x.
- (3) When the memory manager receives a **total_order_broadcast** (*Read*, x) from network:
 - (3a) **no operation**.

Strict consistency / atomic consistency / linearizability



- Why read should participate?



IIT ROORKEE

33

Sequential consistency



- Sequential consistency* (SC) was proposed by Lamport and uses **logical time reference** instead of the global time reference.
- Sequential consistency is specified as follows:
 - The result of any execution is the same as if all operations of the processors were executed in *some* sequential order.
 - The operations of each individual processor appear in this sequence in the local program order.
- In this model, even if two operations from different processors (on the same or different variables) do not overlap in a global time scale, they may appear in reverse order in the common sequential order seen by all the processors.**

IIT ROORKEE

34

Sequential consistency

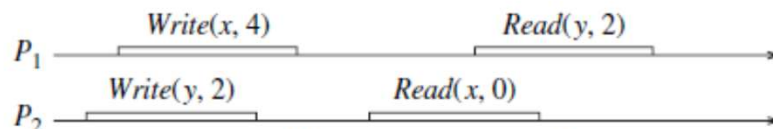


- More formally, a sequence Seq of invocation and response events is sequentially consistent if there is a permutation Seq' of adjacent pairs of corresponding $\langle \text{invoc}, \text{resp} \rangle$ events satisfying:
 - For every variable v , the projection of Seq' on v , denoted Seq'_v , is such that every *Read* (adjacent $\langle \text{invoc}, \text{resp} \rangle$ event pair) returns the most recent *Write* (adjacent $\langle \text{invoc}, \text{resp} \rangle$ event pair) that immediately preceded it.
 - If the response $op1(\text{resp})$ of operation $op1$ at process P_i occurred before the invocation $op2(\text{invoc})$ of operation $op2$ [by process P_i in Seq], then $op1$ (adjacent $\langle \text{invoc}, \text{resp} \rangle$ event pair) occurs before $op2$ (adjacent $\langle \text{invoc}, \text{resp} \rangle$ event pair) in Seq' .

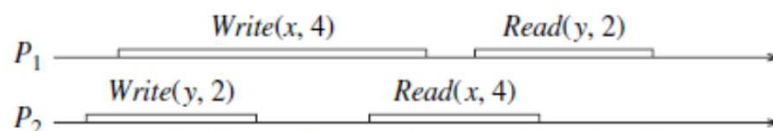
IIT ROORKEE

35

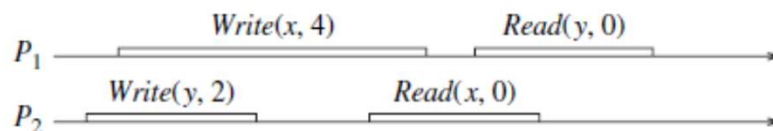
Consistency



(a) Sequentially consistent but not linearizable



(b) Sequentially consistent and linearizable



(c) Not sequentially consistent (and hence not linearizable)

IIT ROORKEE

36